Les structures itératives en langage C (explication simple)

Ce document explique les structures itératives (boucles) du langage C de manière simple, pour une débutante qui découvre la programmation.

1. Le langage C, c'est quoi ?

Le langage C est un langage de programmation qui permet de donner des instructions claires et précises à l'ordinateur. L'ordinateur exécute exactement ce qu'on lui demande, ni plus ni moins.

2. Pourquoi utiliser des boucles ?

Une boucle sert à répéter plusieurs fois une même action sans réécrire le code. Par exemple : afficher des nombres, redemander une valeur, compter, etc. 1 2 3 1 2 3

3. La structure while

La boucle while répète des instructions tant que la condition est vraie.

Exemple : afficher les nombres de 1 à 10.

int vient de **integer** (nombre entier).

La variable I……..

while veut dire **"tant que"**.

```
int i = 1;
while(i <= 10)
{
  printf("%d ", i);
  i++;
}
```

◈ Les deux lignes à expliquer

```
printf("%d ", i);
```

```
i++;
```

Ces deux lignes sont **le cœur de la boucle**.
Sans elles ➜ le programme ne sert à rien 😄

1️⃣ printf("%d ", i);

◆

C’est quoi printf ?

printf est une **fonction d’affichage**.

👉 Elle sert à **montrer quelque chose à l’écran**.

Pense à printf comme :

🗣️ *la bouche de l’ordinateur*

◆ **Que veut dire "%d " ?**

- %d ➜ signifie **nombre entier**
- l’espace " " ➜ ajoute un petit espace après le nombre (pour que ce soit lisible)

Donc :

"%d "

veut dire :

“Affiche un nombre entier, puis laisse un espace”

◆ **Et i alors ?**

i est la **valeur à afficher**.

👉 Si i = 1
➡️ l'écran affiche :

1

👉 Si i = 5
➡️ l'écran affiche :

5 25

🔶 Traduction complète en français

printf("%d ", i);

💬 *"Affiche la valeur actuelle de i à l'écran."*

2️⃣ i++;

⚠️ **Ligne CRUCIALE**.
Sans elle, le programme casse.

🔶 Que veut dire i++ ?

i++ signifie :

i = i + 1;

👉 On ajoute **1** à la valeur de i.

Exemples :

- si i = 1 ➜ après i++, i = 2
- si i = 9 ➜ après i++, i = 10

🔶 Pourquoi on en a besoin ?

Parce que i est le **compteur**.

👉 Il permet de :

- avancer
- se rapprocher de la fin
- sortir de la boucle

Sans i++ :

- i ne change jamais
- la condition reste vraie
- la boucle tourne **à l'infini** 😵‍💫

3️⃣ Les deux lignes ensemble : ce qu'elles font vraiment

```
printf("%d ", i);
i++;
```

L'ordinateur fait ceci :

1. J'affiche la valeur de i
2. J'augmente i
3. Je reviens vérifier la condition du while

Encore et encore 🔁

🔄 Exemple pas à pas

Début :

i = 1;

Étape	Action	Valeur de i
1	printf	affiche 1
2	i++	i devient 2
3	printf	affiche 2
4	i++	i devient 3

Étape	Action	Valeur de i
...	...	...
fin	i = 11	stop

🧠 Image mentale pour débutante

Imagine quelqu'un qui compte à voix haute :

"Je dis mon nombre…
puis je passe au suivant."

- dire le nombre ➜ printf
- passer au suivant ➜ i++

🧭 À retenir ABSOLUMENT

- printf ➜ affiche à l'écran
- %d ➜ nombre entier
- i ➜ valeur affichée
- i++ ➜ fait avancer le compteur
- sans i++ ➜ programme bloqué ❌

Si tu veux, prochain niveau :

- pourquoi on met parfois i--
- différence entre i++ et ++i
- ou un **exercice ultra simple** pour vérifier qu'elle a compris

Dis-moi 👇

4. La structure do...while

La boucle do...while fonctionne comme while, mais elle s'exécute au moins une fois, même si la condition est fausse.

Exemple :

```
int i = 11;
do
{
  printf("%d ", i);
  i++;
}
while(i <= 10);
```

moi-meme Mon exemple de comprehension

```
int i= 14

 do
{
  printf("%d ", i);
  i++;
}
while(i <= 12);
```

Le programme à expliquer

```
int i = 11;
do
{
  printf("%d ", i);
  i++;
```

```
}
while(i <= 10);
```

---

1 int i = 11;

- int signifie **nombre entier**
- i est le **nom de la variable**
- 11 est la **valeur de départ**

👉 On crée donc une variable appelée i qui contient le nombre **11**.

💬 *Imagine une boîte avec le nombre 11 dedans.*

---

2 do { … }

do veut dire **“fais”**.

👉 Ici, l’ordinateur reçoit cet ordre :

“Exécute ce qui est à l’intérieur des accolades { }.”

⚠ Très important :

- **Aucune condition n’est testée ici**
- Le programme **entre obligatoirement** dans le bloc

---

3 printf("%d ", i);

- printf sert à **afficher quelque chose à l’écran**
- %d signifie **nombre entier**
- i est la valeur à afficher

👉 Comme i = 11, l’ordinateur affiche :

11

💬 *C’est l’ordinateur qui parle et qui dit : « 11 ».*

---

4 i++;

- i++ signifie :
  i = i + 1

Donc :

- avant : i = 11
- après : i = 12

On avance d'un pas.

---

5 while(i <= 10);

Maintenant seulement, l'ordinateur pose la question :

"Est-ce que i est inférieur ou égal à 10 ?"

Ici :

- i = 12
- 12 <= 10 ✕ **faux**

Donc :
la boucle s'arrête immédiatement

Ce qui se passe réellement (pas à pas)

Étape	Valeur de i	Action
départ	11	entrée dans do
affichage	11	printf
augmentation	12	i++
test	12 ≤ 10 ✕	arrêt

Résultat final affiché :

11

🤔 Pourquoi ce programme affiche quand même 11 ?

Parce que :

🔑 La boucle do...while s'exécute toujours au moins une fois

Même si la condition est fausse au départ.

🆚 À comparer (très important pour débutante)

Si on avait écrit :

```
while(i <= 10)
{
  printf("%d ", i);
  i++;
}
```

👉 Résultat :

- **Rien ne s'affiche**
- Parce que la condition est testée **avant**

🎯 Phrase simple à retenir

👉 **"do...while exécute d'abord, puis vérifie après."**

🎯 Conclusion pour la débutante

- int → nombre entier
- i → variable
- do → exécuter au moins une fois
- printf → afficher
- i++ → avancer
- while → décider si on recommence

5. La structure for

La boucle for est utilisée quand on connaît à l'avance le nombre de répétitions.

Exemple :

```
for(int i = 1; i <= 10; i++)
{
  printf("%d ", i);
}
```

LA STRUCTURE for — EXPLICATION MOT PAR MOT

Regarde bien cette ligne :

for(int i = 1; i <= 10; i++)

Elle contient **3 parties**, séparées par des **points-virgules (;)**.

---

1 int i = 1 → LE DÉPART

👉 Ici, on dit à l'ordinateur :

- int ➜ je crée un **nombre entier**
- i ➜ c'est le **nom du compteur**
- = 1 ➜ il commence à **1**

💬 *"Je commence à compter à partir de 1."*

⚠️ Cette partie s'exécute **UNE SEULE FOIS**, au début.

---

2 i <= 10 → LA CONDITION

👉 L'ordinateur se pose cette question :

"Est-ce que i est inférieur ou égal à 10 ?"

- ✅ Oui ➜ on entre dans la boucle
- ❌ Non ➜ on sort de la boucle

💬 *"Tant que je n'ai pas dépassé 10, je continue."*

3️⃣ i++ → L'AVANCEMENT

👉 i++ signifie :

i = i + 1;

Donc :

- 1 devient 2
- 2 devient 3
- 3 devient 4
- etc.

💬 *"J'avance d'un pas."*

⚠️ Cette partie s'exécute **À LA FIN de chaque répétition**.

🧩 MAINTENANT LE BLOC { }

```
{
  printf("%d ", i);
}
```

👉 Tout ce qu'il y a **entre les accolades** est **répété**.

🔷 printf("%d ", i);

- printf ➜ affiche à l'écran
- %d ➜ un nombre entier
- i ➜ la valeur à afficher

👉 L'ordinateur affiche la valeur actuelle de i.

🔄 DÉROULEMENT COMPLET (PAS À PAS)

Début :

int i = 1;

Tour	Valeur de i	Condition i <= 10	Affichage
1	1	vraie	1
2	2	vraie	2
3	3	vraie	3
4	4	vraie	4
5	5	vraie	5
6	6	vraie	6
7	7	vraie	7
8	8	vraie	8
9	9	vraie	9
10	10	vraie	10
11	11	fausse ❌	STOP

👉 Résultat affiché :

1 2 3 4 5 6 7 8 9 10

🧠 IMAGE MENTALE (TRÈS IMPORTANTE)

Imagine quelqu'un qui compte :

"Je commence à 1
Tant que je n'ai pas dépassé 10
Je dis mon nombre
Puis je passe au suivant"

- commencer ➜ int i = 1
- règle ➜ i <= 10

- dire le nombre → printf
- avancer → i++

🌟 PHRASE MAGIQUE À RETENIR

👉 **“La boucle for rassemble le début, la condition et l’avancement au même endroit.”**

🆚 POUR COMPARER (IMPORTANT)

Ce for :

```
for(int i = 1; i <= 10; i++)
```

fait exactement la même chose que :

```
int i = 1;
while(i <= 10)
{
  printf("%d ", i);
  i++;
}
```

👉 Mais for est :

- plus clair
- plus propre
- plus utilisé quand on connaît le nombre de répétitions

🎯 CONCLUSION POUR DÉBUTANTE

- for → boucle
- int i = 1 → point de départ
- i <= 10 → règle pour continuer
- i++ → avancer
- { } → ce qui est répété

- printf ➜ afficher

6. Résumé

- while : on teste la condition avant d'exécuter
- do...while : on exécute au moins une fois
- for : tout est regroupé au même endroit